



Architecture Recovery Document — Chamilo LMS 2.0

Course: Software Reengineering

Semester Project: Software Analysis, Re-engineering & Modernization Plan

Target System (Upstream): Chamilo LMS — chamilo/chamilo-lms

Recovered Views: Logical, Physical/Deployment, Process/Runtime

Group – 2

- 1) Haffiz-Ud-Din
- 2) Musharraf Ali
- 3) Akmal Hussain
- 4) Hasrat Ali
- 5) Anoosha Ali
- 6) Arooj Ahmed

School of Computing and Emerging Technologies KIU GILGIT

Tabel Of Contents

1. Executive Summary	4
2. Scope, Assumptions, and Constraints	4
2.1 Scope.....	4
2.2 Out of Scope.....	5
2.3 Key Constraints.....	5
3. Methodology	5
3.1 Evidence Sources.....	5
3.2 Recovery Strategy	6
3.3 Notation and Artifact Storage.....	6
4. Logical View	6
4.1 Context Diagram.....	6
4.1.1 Purpose.....	6
4.1.2 Interpretation	7
4.1.3 Diagram	7
4.2 Component Diagram	8
4.2.1 Purpose.....	8
4.2.2 Recovered Responsibilities	8
4.2.3 Diagram	9
4.3 Data / ER Diagram.....	9
4.3.1 Purpose.....	9
4.3.2 Recovered Data Concepts.....	10
4.3.3 Diagram	10
5. Physical / Deployment View	10
5.1 Deployment/Container Diagram	10
5.1.1 Purpose.....	10
5.1.2 What this supports in the project	11
5.1.3 Diagram	11
6. Process / Runtime View	12
6.1 Activity Diagram — Overall Service-Providing Behavior	12
6.1.1 Purpose.....	12

6.1.2 Recovered runtime decisions	12
6.1.3 Diagram	13
6.2 Sequence Diagram — Login + JWT Token Flow.....	14
6.2.1 Purpose.....	14
6.2.2 Key security controls (Recovered).....	14
6.2.3 Diagram	14
6.3 Sequence Diagram — Document Upload Flow	15
6.3.1 Purpose.....	15
6.3.2 Why this flow is architecturally important.....	15
6.3.3 Diagram	16
6.4 Sequence Diagram — Course Creation + Events + Plugin Subscribers.....	16
6.4.1 Purpose.....	16
6.4.2 Why this matters for modernization	17
6.4.3 Diagram	17
7. Recovered Architectural Decisions & Patterns	18
7.1 Architectural style	18
7.2 Key patterns	18
7.3 Architectural constraints.....	18
8. Key Risks, Limitations, and Gaps	18
8.1 Risks for the semester project.....	18
8.2 Limitations of this recovered model	19
8.3 Gaps to close during implementation phase.....	19
9. Appendix	19
9.1 Glossary	19
9.2 References	20

1. Executive Summary

This document presents an **architecture recovery** for *Chamilo LMS 2.0* based on repository inspection, configuration analysis, and reconstruction of runtime behavior. The recovered architecture is expressed using three complementary views:

- **Logical View:** What the system *is* — core modules, responsibilities, internal dependencies, key domain entities.
- **Physical / Deployment View:** Where the system *runs* — infrastructure services, runtime dependencies, storage topology, external integrations.
- **Process / Runtime View:** How the system *behaves* — request lifecycle, routing decisions, authentication modes, persistence/storage steps, and extension via plugins/events.

The purpose of this recovery is academic and practical:

- to create a credible understanding of Chamilo's current architecture,
- to identify modernization opportunities and constraints,
- to support refactoring and re-engineering activities,
- to prepare a final live demonstration aligned with recovered flows (login, upload, course creation).

2. Scope, Assumptions, and Constraints

2.1 Scope

This architecture recovery covers the major application layers and cross-cutting concerns that influence modernization work:

- **Backend runtime:** PHP 8.2+, Symfony 6.4, Doctrine ORM, Doctrine migrations
- **API layer:** API Platform 3, custom API actions, and API Platform providers/processors
- **Frontend:** Vue 3 SPA areas, PrimeVue component usage, Webpack Encore build pipeline, and legacy JS bundles
- **Legacy layer:** legacy PHP tools and pages under `public/main/*` and the compatibility approach to keep them working
- **Plugins:** plugin directories under `public/plugin/*`, UI region injection, and event-subscriber integration

- **Infrastructure dependencies:** MariaDB/MySQL database, Flysystem file storage (local/cloud), optional Redis/Messenger
- **External integrations (conceptual):** BigBlueButton, Zoom, OnlyOffice, LTI tool providers, email sending

2.2 Out of Scope

- Full catalog of all legacy tools and their internal script dependencies
- Full database schema (Chamilo maps hundreds of entities/tables)
- Complete inventory of every API resource and every Vue route
- Deep performance profiling (covered in a separate testing/validation report)

2.3 Key Constraints

The following constraints strongly shape modernization:

1. **Hybrid architecture:** A modern Symfony/Vue architecture coexists with a large legacy PHP layer.
2. **Backward compatibility:** Legacy pages/tools still require fixed asset paths and early-loaded scripts.
3. **Multi-tenant support:** “Access URLs” influence plugin enablement and settings resolution.
4. **Storage abstraction:** Files must work across local disk and cloud backends via Flysystem.
5. **Security model complexity:** Web interface and API use different mechanisms (session vs JWT) with shared authorization decisions (roles/voters).

3. Methodology

3.1 Evidence Sources

Architecture was recovered using:

- Repository structure inspection (src/, public/, assets/, config/)
- Runtime entry points (public/index.php, root index.php, installer path)
- Dependency manifests (composer.json, package.json)
- Environment configuration template (.env.dist)

- Build config (webpack.config.js) and inferred frontend structure
- Provided developer guide excerpts (plugins, Vue Base components, theming and routing patterns)

3.2 Recovery Strategy

We used a three-step recovery:

1. **Static structure recovery:** identify major subsystems and their boundaries from code layout and configuration.
2. **Dynamic flow recovery:** reconstruct request lifecycles and end-to-end flows from entry points and framework pipelines (Symfony + API Platform + Vue client).
3. **Domain/data recovery:** model core entities required for the chosen scope and typical flows (Users/Courses/Sessions, Resource system, Document example).

3.3 Notation and Artifact Storage

- Diagrams are defined using **PlantUML** (.puml) for repeatability.
- Export each diagram as .svg (preferred for zooming in reports) or .png.
- Diagrams are stored under:
 - diagrams/logical/
 - diagrams/physical/
 - diagrams/process/

4. Logical View

The logical view explains **what Chamilo is made of** and how responsibilities are split across its main subsystems.

4.1 Context Diagram

4.1.1 Purpose

The context diagram establishes:

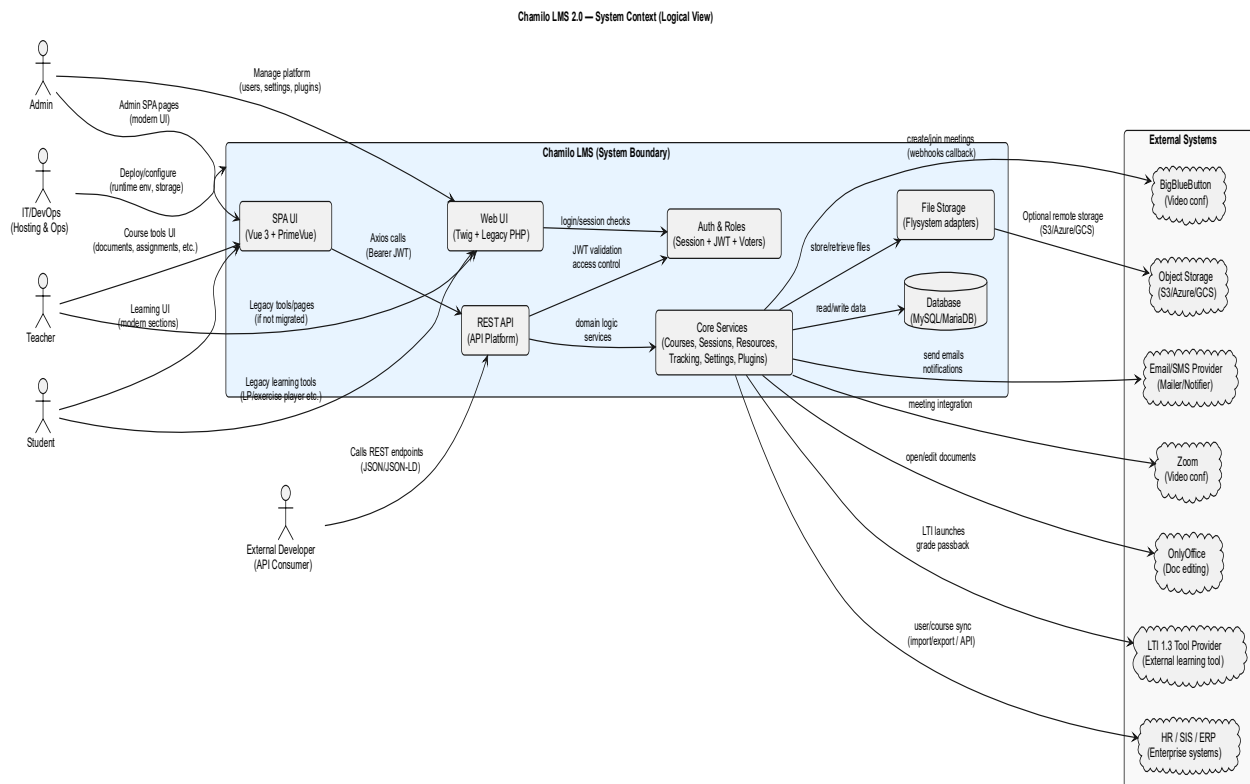
- system boundary (**Chamilo LMS**),
- key human actors (Admin, Teacher, Student),

- external systems (BBB/Zoom/OnlyOffice/LTI/HR-SIS/email/object storage),
- primary interaction channels: web UI, SPA UI, REST API, integrations.

4.1.2 Interpretation

- **Admins** configure the platform (users, settings, plugins, access URLs, themes).
- **Teachers** create and manage course content (documents, assignments, learning paths, activities).
- **Students** consume content and perform learning activities (view documents, take exercises, participate in sessions).
- **External tools** provide conferencing (BBB/Zoom), document editing (OnlyOffice), and LTI interoperability.
- **Storage and mail** are infrastructure dependencies rather than user-facing tools.

4.1.3 Diagram



- Export file: diagrams/logical/context-diagram.svg
- Source file: diagrams/logical/context-diagram.puml

4.2 Component Diagram

4.2.1 Purpose

The component diagram describes the major architectural partitions:

- Symfony bundles (CoreBundle, CourseBundle, LtiBundle)
- API Platform layer (resources, operations, custom actions, providers/processors)
- Frontend (Vue 3 SPA and legacy JS bundles)
- Legacy PHP tooling (public/main/*)
- Plugin system (public/plugin/*)
- Persistence (Doctrine) and storage (Flysystem)

4.2.2 Recovered Responsibilities

CoreBundle

- User management, security roles and voters, platform settings, resource model, administration UI, plugin helpers, integrations.

CourseBundle

- Course-scoped entities (documents, quizzes, learning paths, forums, surveys, attendance, assignments) and course-specific controllers and settings.

LtiBundle

- LTI 1.3 integration (launch flow, tool registration, grade return).

API Layer

- API Platform provides standard REST operations.
- Special operations (uploads, exports, moves) are implemented via **custom controllers** or **state processors/providers**.

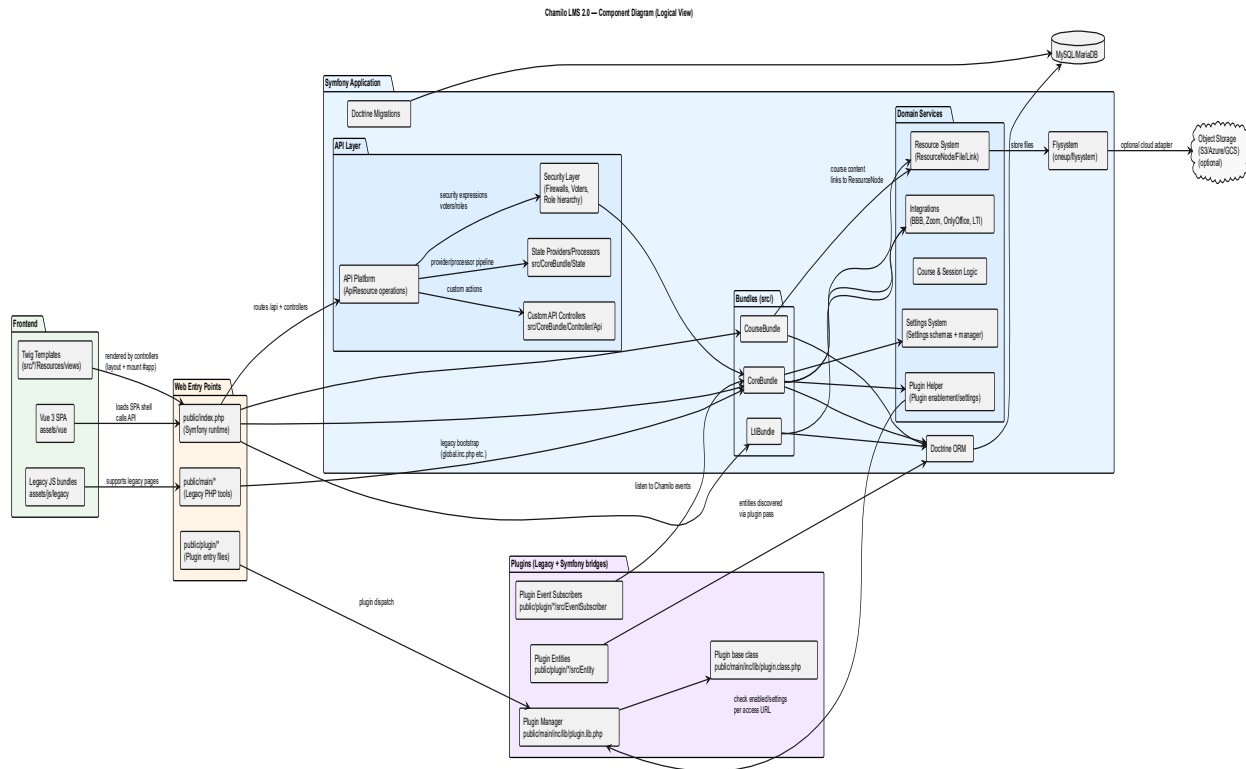
Frontend

- Vue 3 SPA supports modern UI with consistent base components.
- Legacy JS bundles support older tools and pages requiring non-SPA scripts.

Legacy Layer

- Many tools are still available as script-driven modules under public/main/*, executed via a compatibility mechanism to keep them working inside the overall platform.

4.2.3 Diagram



- Export file: diagrams/logical/component-diagram.svg
- Source file: diagrams/logical/component-diagram.puml

4.3 Data / ER Diagram

4.3.1 Purpose

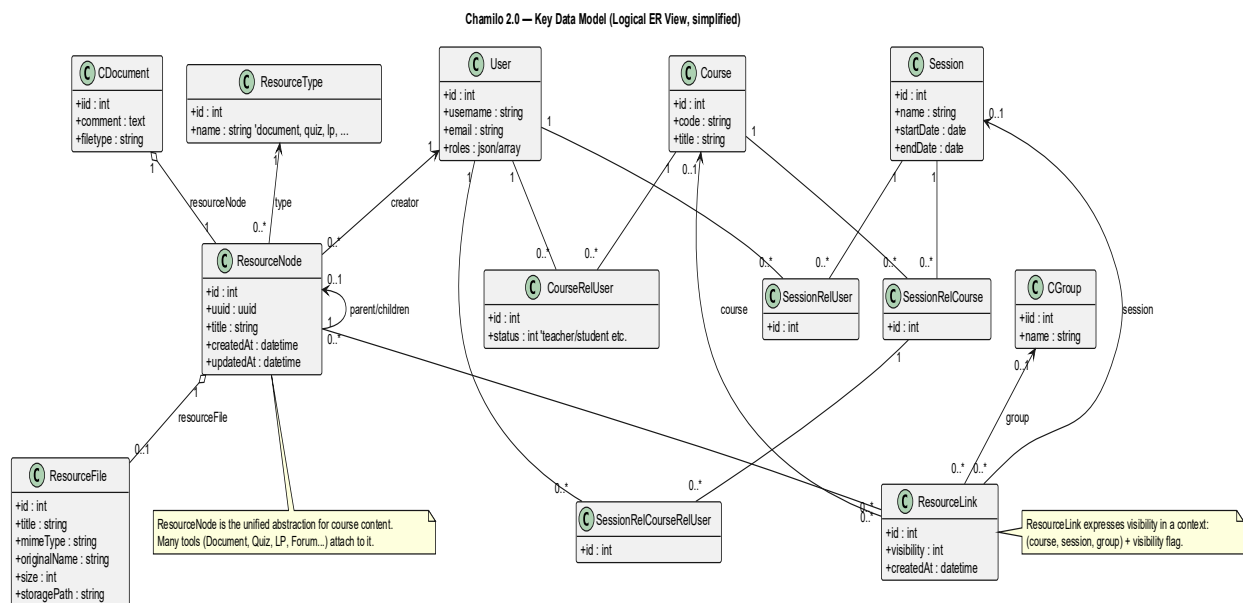
This ER model is a **simplified architecture-recovery data model** focused on:

- authentication and identity (User),
- organizational context (Course, Session),
- enrollment relationships (join entities),
- the “unified content abstraction” (ResourceNode, ResourceFile, ResourceLink, ResourceType),
- a representative course tool entity (CDocument) as an example.

4.3.2 Recovered Data Concepts

1. **Enrollment is modeled via relation entities** (e.g., CourseRelUser, SessionRelCourseRelUser), which store status/role and context.
2. **ResourceNode** is a central abstraction used across many content types.
3. **Visibility is context-dependent** via ResourceLink (course/session/group).
4. Files are abstracted via ResourceFile and stored through Flysystem, enabling local or cloud storage.

4.3.3 Diagram



- Export file: diagrams/logical/data-er-diagram.svg
- Source file: diagrams/logical/data-er-diagram.puml

5. Physical / Deployment View

This view explains how Chamilo is typically deployed and the infrastructure services involved.

5.1 Deployment/Container Diagram

5.1.1 Purpose

Represents a typical local development environment (ideal for reproducible setup):

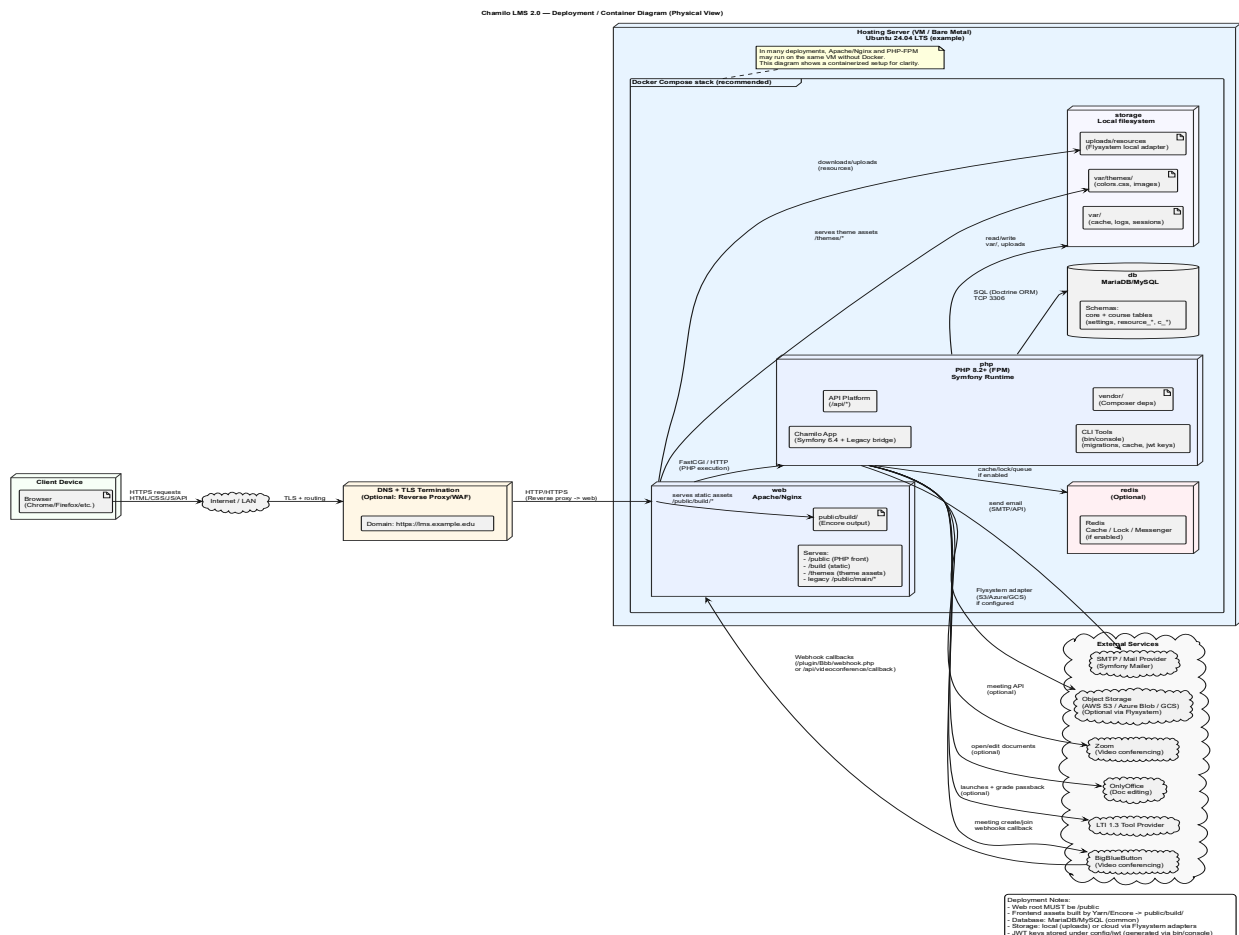
- browser connects to a local host/domain,
- web server serves /public and static /public/build,
- PHP runtime executes Symfony and legacy scripts,
- MariaDB/MySQL stores data,
- local filesystem stores caches/uploads/themes,
- optional Redis is used for cache/locks/queues if enabled.

5.1.2 What this supports in the project

This deployment view supports:

- consistent local setup for all group members,
- stable CI-like reproduction,
- predictable demo environment.

5.1.3 Diagram



- Export file: diagrams/physical/deployment-dev.svg
- Source file: diagrams/physical/deployment-diagram-dev.puml

6. Process / Runtime View

This view captures how Chamilo behaves at runtime and provides evidence for modernization planning.

6.1 Activity Diagram — Overall Service-Providing Behavior

6.1.1 Purpose

This activity diagram models the overall service behavior, including:

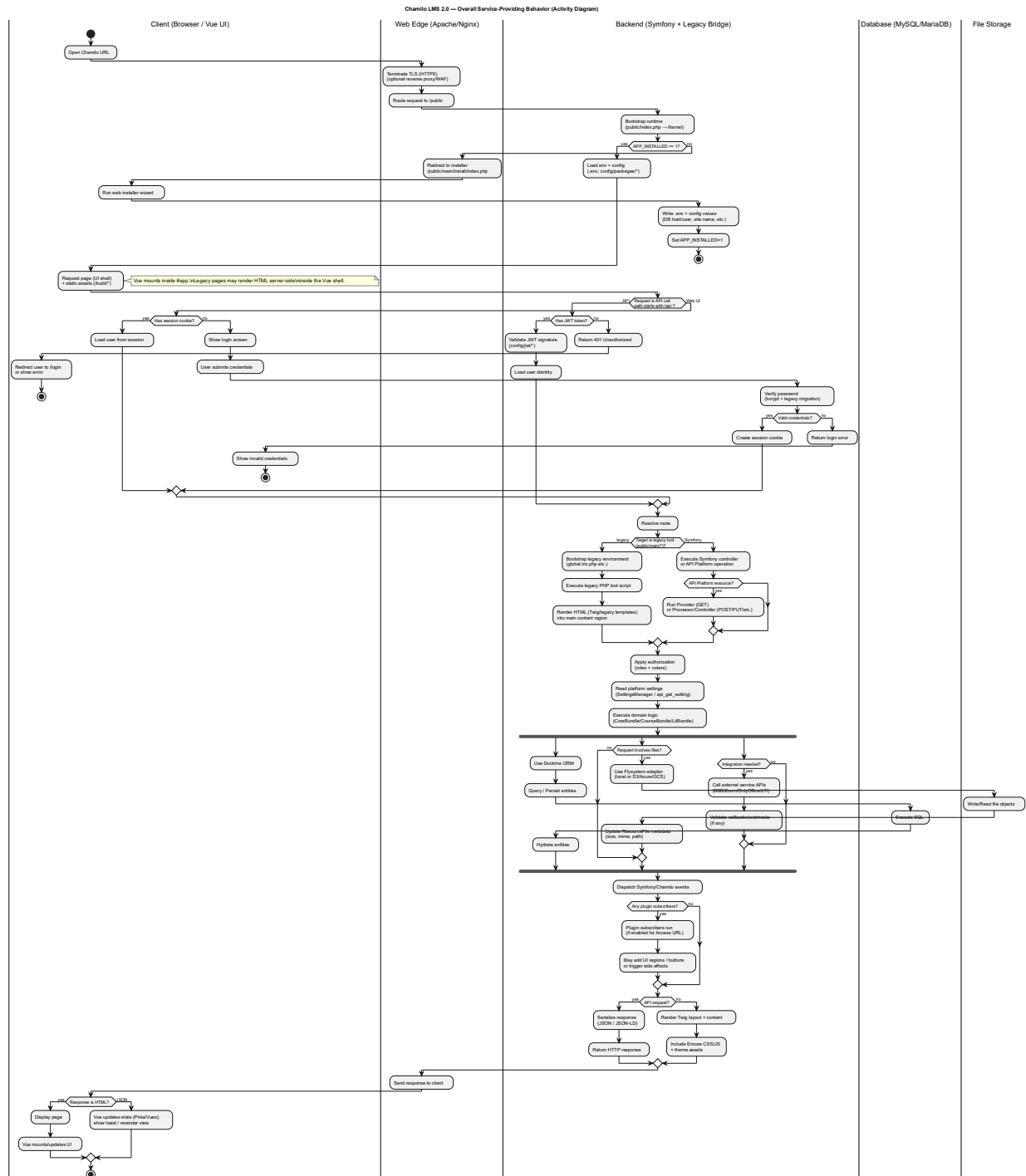
- bootstrapping and install-check (APP_INSTALLED),
- authentication paths (session-based web UI vs JWT-based API),
- routing decisions (legacy tool execution vs Symfony/API Platform execution),
- domain logic execution and persistence,
- file storage through Flysystem,
- event dispatch and plugin subscriber participation,
- response generation (HTML/Twig vs JSON).

6.1.2 Recovered runtime decisions

Key decision points:

1. **Installed vs not installed:** redirect to installer when not installed.
2. **API vs Web request:** /api/* uses JWT firewall; classic pages rely on sessions.
3. **Legacy vs modern route:** some actions are handled in legacy scripts and still rely on legacy JS.
4. **File flow:** some operations require file writes and metadata persistence.
5. **Plugins:** may inject UI or add side effects via events.

6.1.3 Diagram



- Export file: diagrams/process/activity-overall-service-behavior.svg
- Source file: diagrams/process/activity-overall-service-behavior.puml

6.2 Sequence Diagram — Login + JWT Token Flow

6.2.1 Purpose

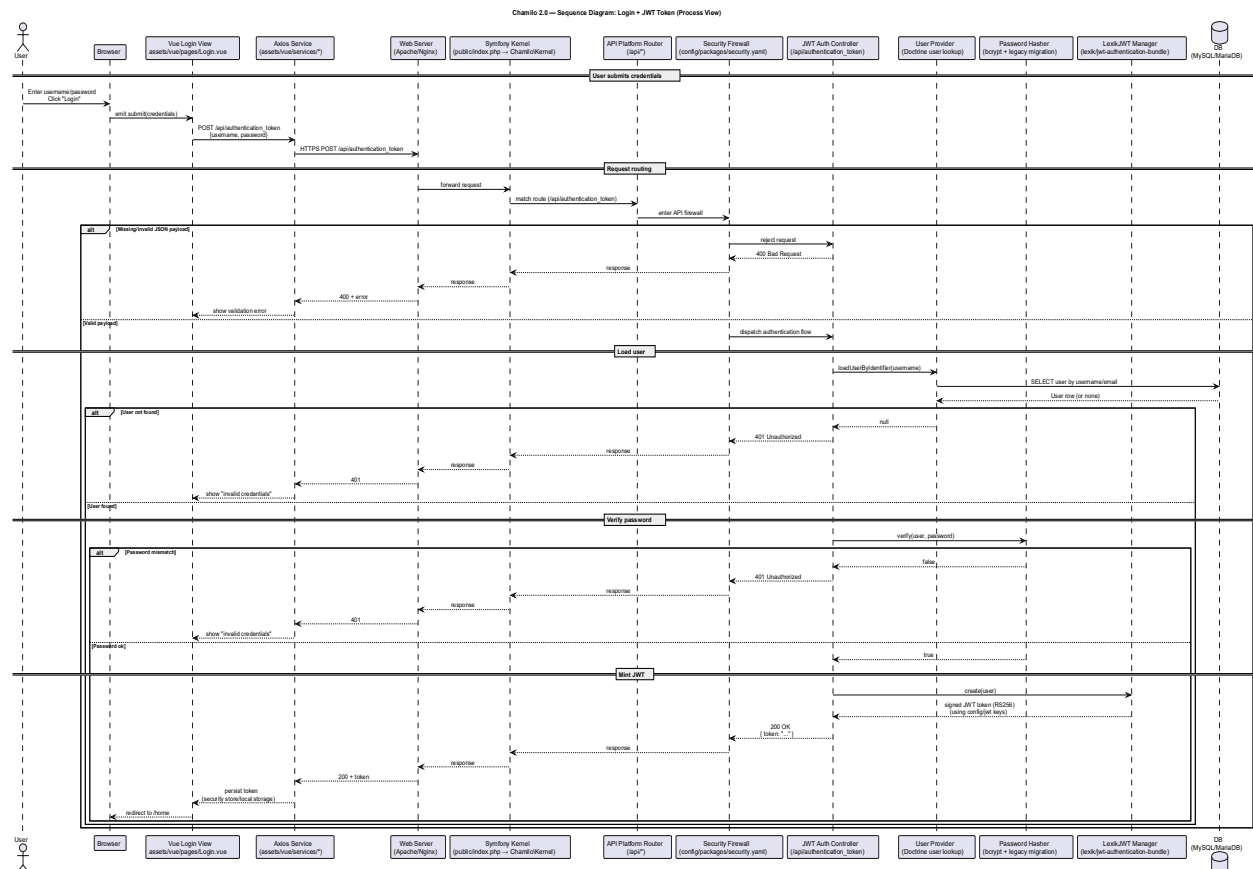
Models the end-to-end login token flow:

- Vue login UI submits credentials to /api/authentication_token,
- Symfony security validates user identity and password,
- JWT token is minted and returned,
- Vue stores token and uses it for authorized API calls.

6.2.2 Key security controls (Recovered)

- API requests require JWT token (Authorization: Bearer <token>).
- Authorization uses role checks and voters for fine-grained access.
- Password hashing uses modern hashing and may support legacy migration (bcrypt + legacy compatibility).

6.2.3 Diagram



- Export file: diagrams/process/seq-login-jwt.svg
- Source file: diagrams/process/seq-login-jwt.puml

6.3 Sequence Diagram — Document Upload Flow

6.3.1 Purpose

Models a full-stack upload workflow:

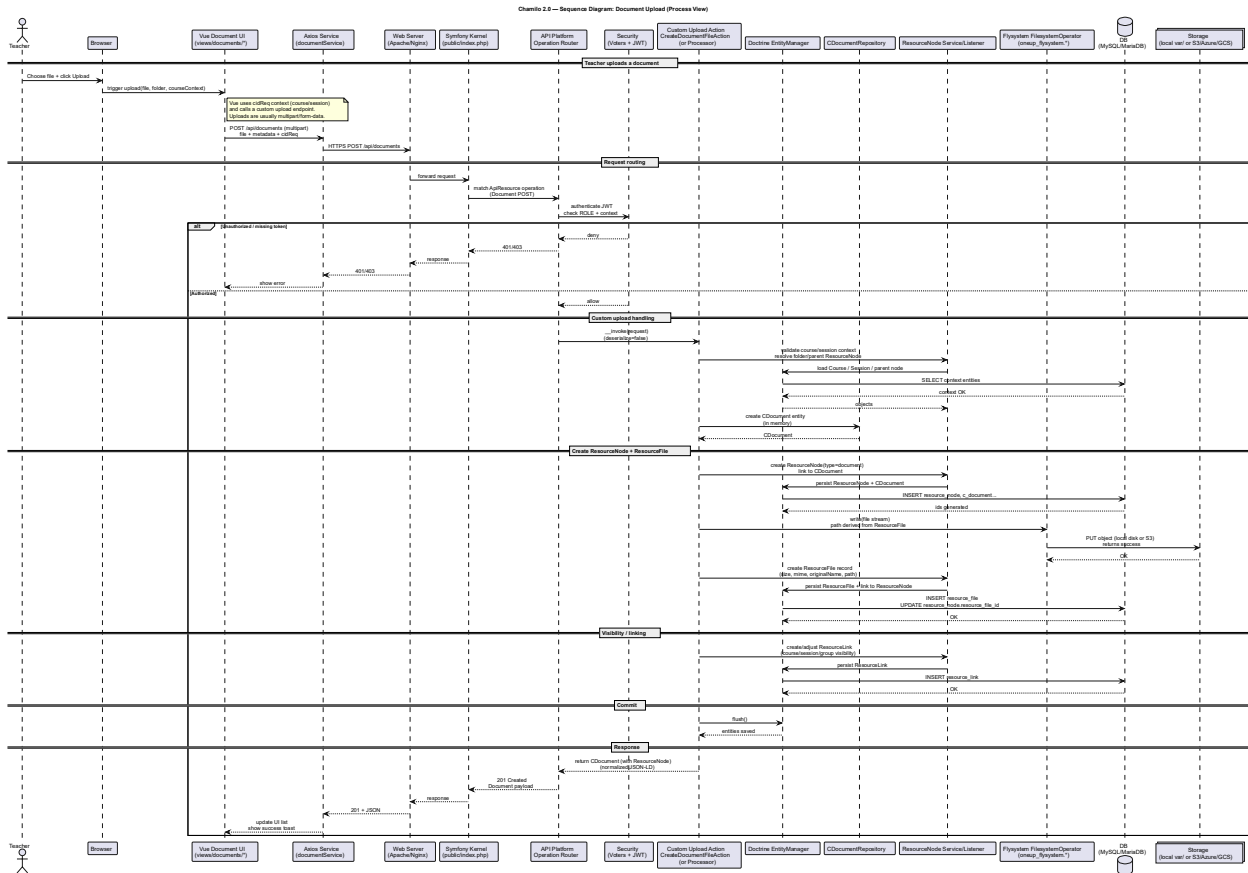
- Vue triggers multipart upload,
- API Platform routes to a custom action/processor,
- database entities are created (CDocument, ResourceNode, ResourceFile, ResourceLink),
- file bytes are written via Flysystem (local disk or object storage),
- response returns document resource payload to update UI.

6.3.2 Why this flow is architecturally important

This scenario crosses:

- UI (Vue) → API (API Platform) → Domain → Persistence (Doctrine/DB) → Storage (Flysystem),
- and shows the purpose of the **ResourceNode abstraction** in Chamilo 2.0.

6.3.3 Diagram



- Export file: diagrams/process/seq-document-upload.svg
- Source file: diagrams/process/seq-document-upload.puml

6.4 Sequence Diagram — Course Creation + Events + Plugin Subscribers

6.4.1 Purpose

Models an administrative workflow with extension points:

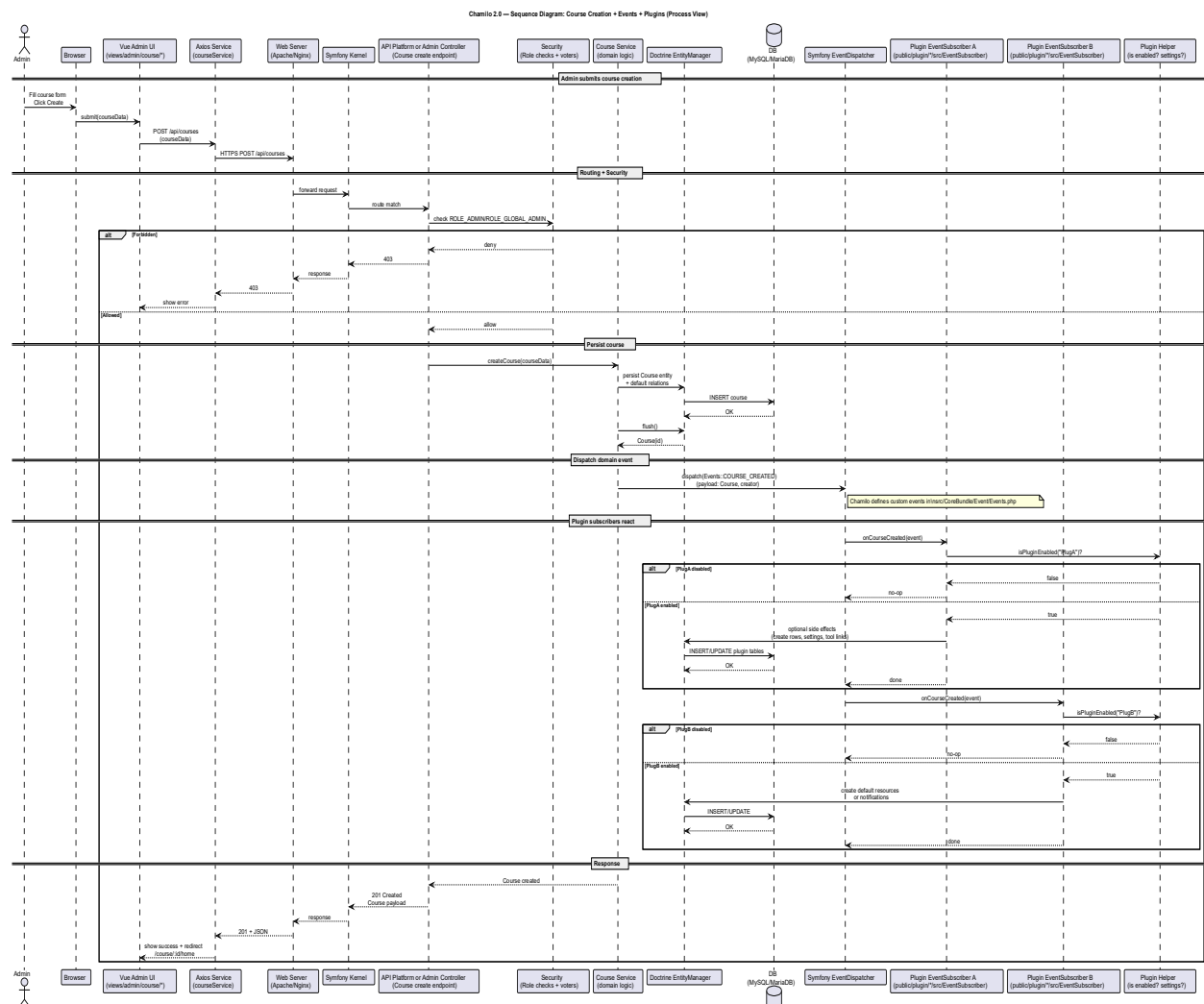
- admin UI submits course creation request,
- backend persists course data,
- Chamilo dispatches a `COURSE_CREATED` event,
- enabled plugin subscribers react (optional side effects),
- response returns course object and UI navigates to course.

6.4.2 Why this matters for modernization

This flow demonstrates:

- core domain service orchestration,
- event-driven extension architecture,
- multi-tenant plugin enablement (per Access URL),
- safe plugin integration patterns (guard by isEnabled()).

6.4.3 Diagram



- Export file: `diagrams/process/seq-course-create-with-plugins.svg`
- Source file: `diagrams/process/seq-course-create-with-plugins.puml`

7. Recovered Architectural Decisions & Patterns

7.1 Architectural style

- **Hybrid modernization:** Symfony + API Platform + Vue coexist with a legacy PHP tool ecosystem.
- **API-driven UI:** modern UI segments behave like an SPA calling a REST API.
- **Domain-centric abstractions:** ResourceNode system unifies content types and enables consistent access control and visibility logic.
- **Extensibility via plugins/events:** Plugins inject UI regions and participate via Symfony event subscribers.
- **Configuration-driven system:** Behavior is heavily influenced by settings schemas, .env variables, and per-AccessURL configuration.

7.2 Key patterns

1. **API Platform providers/processors:** custom logic is attached to standard REST operations.
2. **Action controllers for special cases:** multipart uploads, exports, non-standard responses.
3. **Flysystem abstraction:** decouples storage from filesystem; supports cloud storage.
4. **Event subscribers:** decouple plugin behavior from core.
5. **Frontend base components:** Vue Base* components enforce design consistency across the app.

7.3 Architectural constraints

- Migration must be incremental: legacy tools cannot be removed immediately.
- Build system must continue to provide unhashed assets for legacy pages.
- Storage changes must preserve consistent URLs and permission checks.
- Plugin API stability is important for ecosystem compatibility.

8. Key Risks, Limitations, and Gaps

8.1 Risks for the semester project

- Selecting too many tools/features will increase analysis and implementation effort.

- Environment complexity (PHP extensions, Node/Yarn, DB setup) can delay demos.
- External integrations (BBB/Zoom/OnlyOffice/LTI) may require accounts and network access.

8.2 Limitations of this recovered model

- Simplified ER model does not represent the full schema.
- Plugin behavior varies by enabled plugins and Access URL configuration.
- Actual runtime flows may differ slightly by configuration (messenger transports, caching layers).

8.3 Gaps to close during implementation phase

- Confirm exact endpoints and processors/controllers used for the chosen key flows.
- Validate role/voter rules for the demo scenarios.
- Validate storage adapter behavior (local vs cloud) if changed in modernization work.

9. Appendix

9.1 Glossary

- **API Platform:** Symfony-based framework that generates REST endpoints from entities; Chamilo uses it for /api/*.
- **JWT:** JSON Web Token used for API authentication; minted by LexikJWT bundle.
- **Flysystem:** filesystem abstraction used by Chamilo to store files locally or in cloud object storage.
- **ResourceNode:** core abstraction representing a piece of course content (documents, LP items, etc.).
- **ResourceFile:** metadata record that points to stored file bytes for a resource node.
- **ResourceLink:** context visibility record (course/session/group) for a resource node.
- **Legacy layer:** older Chamilo tools/pages under public/main/*, still used and gradually migrated.

9.2 References

- Upstream repo: [chamilo/chamilo-lms](#) (public GitHub repository)
- [Symfony 6.4 documentation](#) (framework and configuration principles)
- [API Platform 3 documentation](#) (resources, operations, state providers/processors)
- [LexikJWT authentication bundle documentation](#) (JWT auth patterns)
- [Flysystem documentation](#) (local and cloud adapters)

Ensure exported diagrams are committed to the repository so the public URL contains complete artifacts.